

智能系统设计实践

六足机器人避障控制 第一次进展汇报

谭诗弘，黄育民，曹洪琿

武汉大学电子信息学院

2025 年 9 月 29 日



① 主要控制文件

② 控制流程

① 主要控制文件

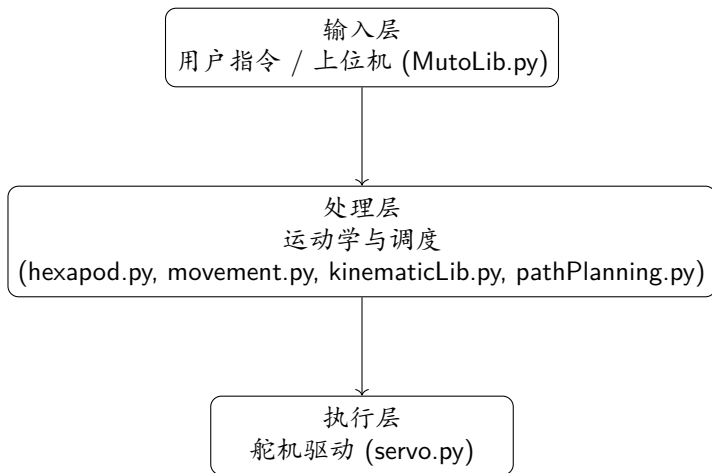
② 控制流程

主要控制文件概览

共 11 个核心模块：

- ① base.py：基础坐标类与腿尖坐标系
- ② config.py：配置参数与预定义步态轨迹
- ③ hexapod.py：六足机器人主控制类
- ④ kinematicLib.py：运动学核心（逆运动学、轨迹生成、坐标变换）
- ⑤ leg.py：单腿控制类，负责逆运动学与舵机控制
- ⑥ movement_paths.py：预定义步态轨迹数据库
- ⑦ movement.py：高层动作控制器（模式切换、轨迹调度）
- ⑧ movement_table.py：静态动作路径表
- ⑨ MutoLib.py：机器人与上位机通信接口
- ⑩ pathPlanning.py：路径规划模块
- ⑪ servo.py：舵机控制模块

控制流程简化图



关键模块重点

在 11 个模块中，以下 5 个最为关键：

- hexapod.py：整体协调
- movement.py：动作调度
- kinematicLib.py：运动学计算
- pathPlanning.py：路径规划
- servo.py：舵机执行

后续将对这些模块进行详细介绍。

Hexapod 主控制类 (hexapod.py)

- 管理六条腿 (RealLeg 实例)
- 接收高层运动指令 (如 `move(x,y,z)`)
- 调用 `movement` 模块生成轨迹点
- 分配轨迹点给各条腿, 保持协调与平衡
- 关键作用: 保证三条腿支撑、三条腿摆动, 实现三角步态

动作调度器 (movement.py)

- 定义动作模式：待机、前进、旋转、攀爬等
- 核心接口：
 - `set_mode()`：切换模式
 - `set_speed()`：调整速度
 - `next()`：取出下一组轨迹点
- 输入：用户动作指令（模式 + 速度）
- 输出：下一步六条腿的 (x,y,z) 坐标
- 特点：结合 `movement_table`（静态路径）和 `pathPlanning`（动态路径）

运动学计算 (kinematicLib.py)

- 逆运动学：足端坐标 $\rightarrow$ 关节角度
- 轨迹生成器：半圆/椭圆路径
- 坐标变换：三维旋转矩阵
- 验证：检查轨迹是否在机械允许范围内
- 地位：是“数学大脑”，确保动作正确

路径规划 (pathPlanning.py)

- 输入：方向、速度、角度
- 输出：六条腿的完整步态轨迹
- 特点：
 - 半圆摆动：平滑抬腿
 - 直线支撑：提供推进力
 - 三角步态：保持平衡
- 作用：生成动态动作路径，供 movement 调度

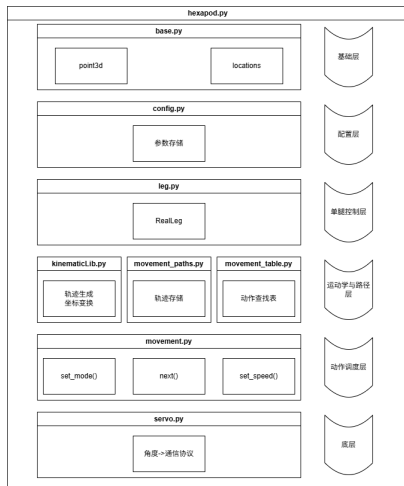
舵机执行层 (servo.py)

- 输入：逆运动学角度与执行时间
- 角度映射：理论角度 $\rightarrow$ 舵机实际角度
- 指令生成：封装通信协议数据帧
- 执行：通过串口下发到舵机
- 地位：是“肌肉”，将计算结果落实为实际动作

① 主要控制文件

② 控制流程

控制流程图



基础层

base.py

定义了 3D 点 (point3d) 和腿尖位置集合 (locations), 作为六足机器人所有运动的几何基础。

- point3d: 描述一个点 (x,y,z) , 支持加减乘法。
- locations: 保存 6 条腿的足尖坐标, 方便批量操作。

提供了坐标系的数据结构, 便于后续的运动轨迹和逆运动学操作。

配置层

config.py

存放机器人硬件和运动学的固定参数。

- 关节长度、安装角度、腿的初始坐标。
- 默认待机位置。
- 预定义的步态轨迹路径点。

通过这些参数计算每条腿运动的范围和姿态。

单腿控制层

leg.py → RealLeg

- 将每条腿视为一个对象，将全局坐标系下的目标点转化为腿的局部坐标系。
- 调用 kinematicLib.py 计算舵机角度
- 调用 servo.py 将角度转化为 PWM 命令

负责具体的单腿控制部分。

运动学与路径层

kinematicLib.py

负责逆运动学操作，将三维点转换为舵机角度。同时生成半圆/椭圆腿尖路径，并处理旋转矩阵的坐标变换。

movement_paths.py

存储不同运动模式的轨迹（如前进、后退、旋转等），每个轨迹有 20 个时间步构成，每一步包含六条腿的三维坐标。

movement_table.py

存储一个动作查找表，将各种运动模式映射到对应的路径点序列，避免每次都实时计算轨迹，加快执行速度。

动作调度层

movement.py

作为动作控制器，负责管理运动模式和路径

- set_mode(): 切换机器人的工作模式。
- set_speed(): 调整机器人的运动速度。
- next(): 取出下一步的运动坐标，供六条腿执行。

其决定机器人下一时刻的运动状态。

底层

servo.py

负责将运动数据转化为舵机通信协议数据帧，通过串口等发送到实际硬件，驱动硬件执行。

负责真实动作的运行。

六足整体协调

hexapod.py → Hexapod 类

总体协调各个模块的工作。

- 初始化六条腿 (RealLeg)。
- 接受用户的运动指令 (move())。
- 调用 movement.py 生成轨迹点。
- 将轨迹点分配给各条腿，调用 leg.move_tip()。
- 保持整体的平衡和同步。

控制流程总结

- ① 用户输入（例如 `hexapod.move(10,0,0)` → 前进 10cm）。
- ② `movement.py` 根据输入调用 `movement_table` / `pathPlanning` 生成完整轨迹点。
- ③ `hexapod.py` 轮询调用 `movement.next()` → 取出 6 条腿的坐标。
- ④ `leg.py` 将目标点 → 逆运动学角度。
- ⑤ `servo.py` 把角度 → 舵机指令。
- ⑥ 舵机动作，机器人移动。

始终保持三条腿支撑、三条腿摆动的三角步态，从而保证平衡。

Thanks!